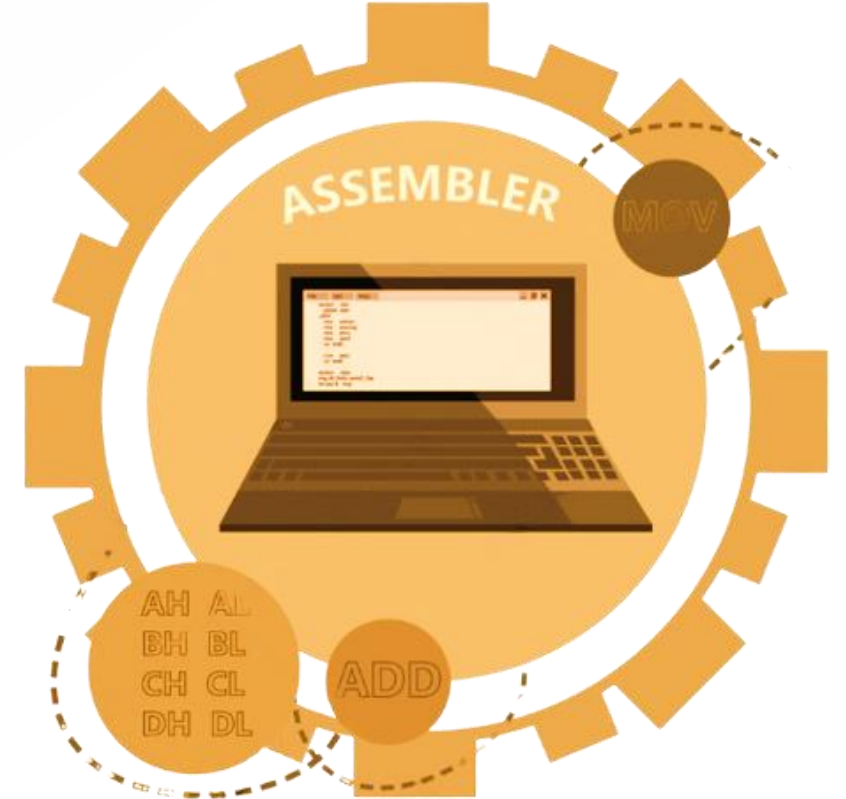


Lecture # 1

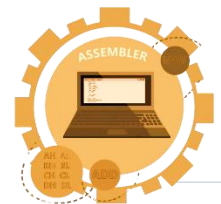
Assembly Language

Engr. Fiaz Khan



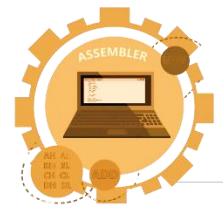
Today's Agenda

- ✓ The components of a Microcomputer System
- ✓ Instruction Execution
- ✓ I/O Devices
- ✓ Programming Languages
- ✓ Assembly Language Program



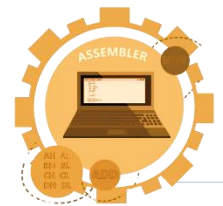
Microcomputer System Components:

- System unit
- Keyboard
- Display screen
- Disk drives



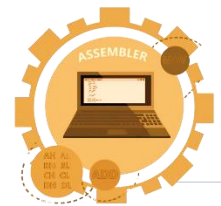
System Unit:

- Often referred to as "the computer"
- Houses circuit boards of the computer



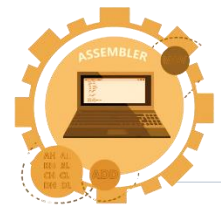
I/O Devices (Input/ Output Devices or Peripherals): Unit:

- Keyboard, display screen, and disk drives
- Perform input/output operations for the computer



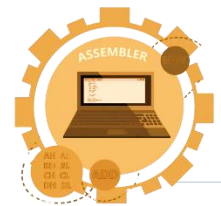
Integrated-Circuit (IC) Chips:

- Used in constructing computer circuits
- Contain hundreds or thousands of transistors



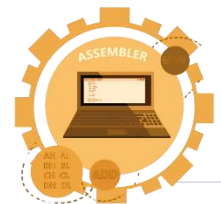
IC Circuits and Digital Signals:

- Operate on discrete voltage signal levels (high and low voltage)
- Represented by symbols 0 and 1 (bits)
- Symbols are called binary digits or bits



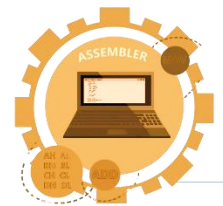
Information Representation:

- All information processed by the computer is represented by strings of 0's and 1's (bit strings) Operate on discrete voltage signal levels (high and low voltage)
- Represented by symbols 0 and 1 (bits)
- Symbols are called binary digits or bits



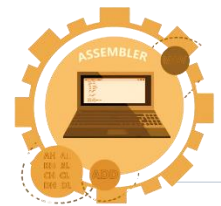
Computer Circuits Functionality:

- Three main parts:
- Central Processing Unit (CPU)
- memory circuits,
- 1/0 circuits



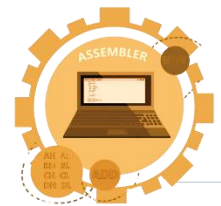
Microcomputer CPU:

- Single-chip microprocessor acts as the CPU
- CPU is the brain of the computer, controlling all operations



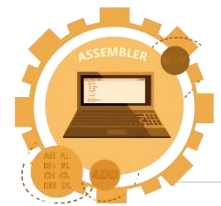
System Board (Motherboard):

- Main circuit board inside the system unit
- Contains microprocessor and memory circuits
- Expansion slots for additional circuit boards (add-in boards or add-in cards)



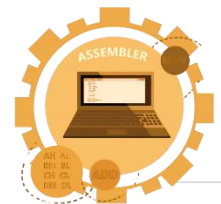
1/O Circuits:

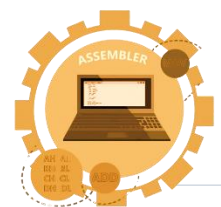
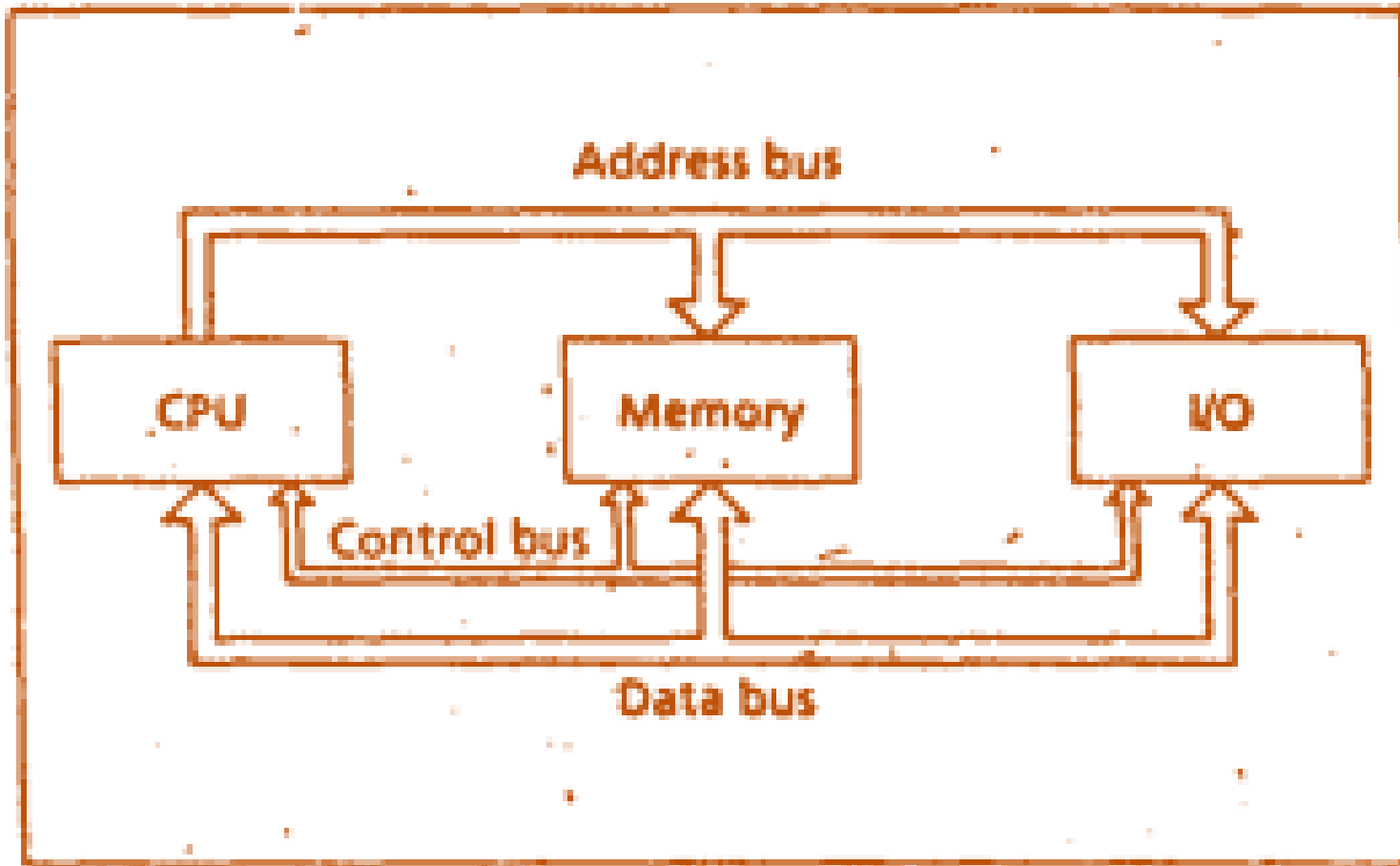
- Usually located on add-in cards
- Used for communication with input/output devices



Memory Organization:

- Information processed by the computer is stored in memory
- Memory circuit element stores one bit of data
- Memory circuits organized into groups (bytes), with eight bits in a byte

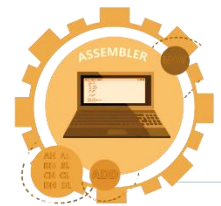






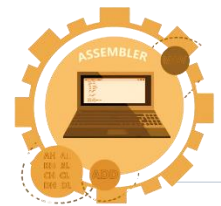
Memory Byte Identification:

- Each memory byte has a number called its address, like a house's street address
- A string of eight bits (byte) is identified by its address



Addressing in Memory:

- Address depends on the processor (e.g., Intel 8086 uses a 20-bit address, Intel 80286 uses a 24-bit address)
- The number of bits in the address determines the number of bytes accessible by the processor.

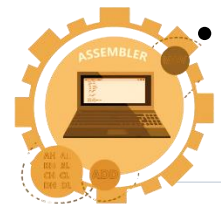


Example:

- Suppose a processor uses 20 bits for an address. How many memory bytes can be accessed?“

Solution:

- To determine the number of memory bytes that can be accessed with a given number of address bits, you can use the formula:
- Number of Memory Bytes= $2^{\text{Number of Address Bits}}$
- In this case, the processor uses 20 bits for an address, so the calculation would be:
- Number of Memory Bytes= 2^{20}
- Let's compute this:
- Number of Memory Bytes= $2^{20}=1,048,576$
- Therefore, with a 20-bit address, the processor can access 1,048,576, memory bytes.
- Thus a 20-bit address can be used to address 1 megabyte or 1 MB.



Bit Positions:

- Shown in Figure 1.4 for a microcomputer word and a byte.
- Numbered from right to left, starting with 0.

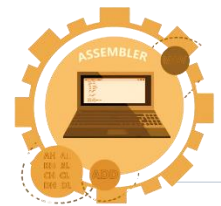
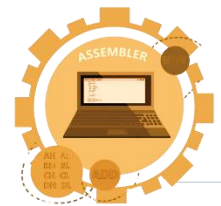
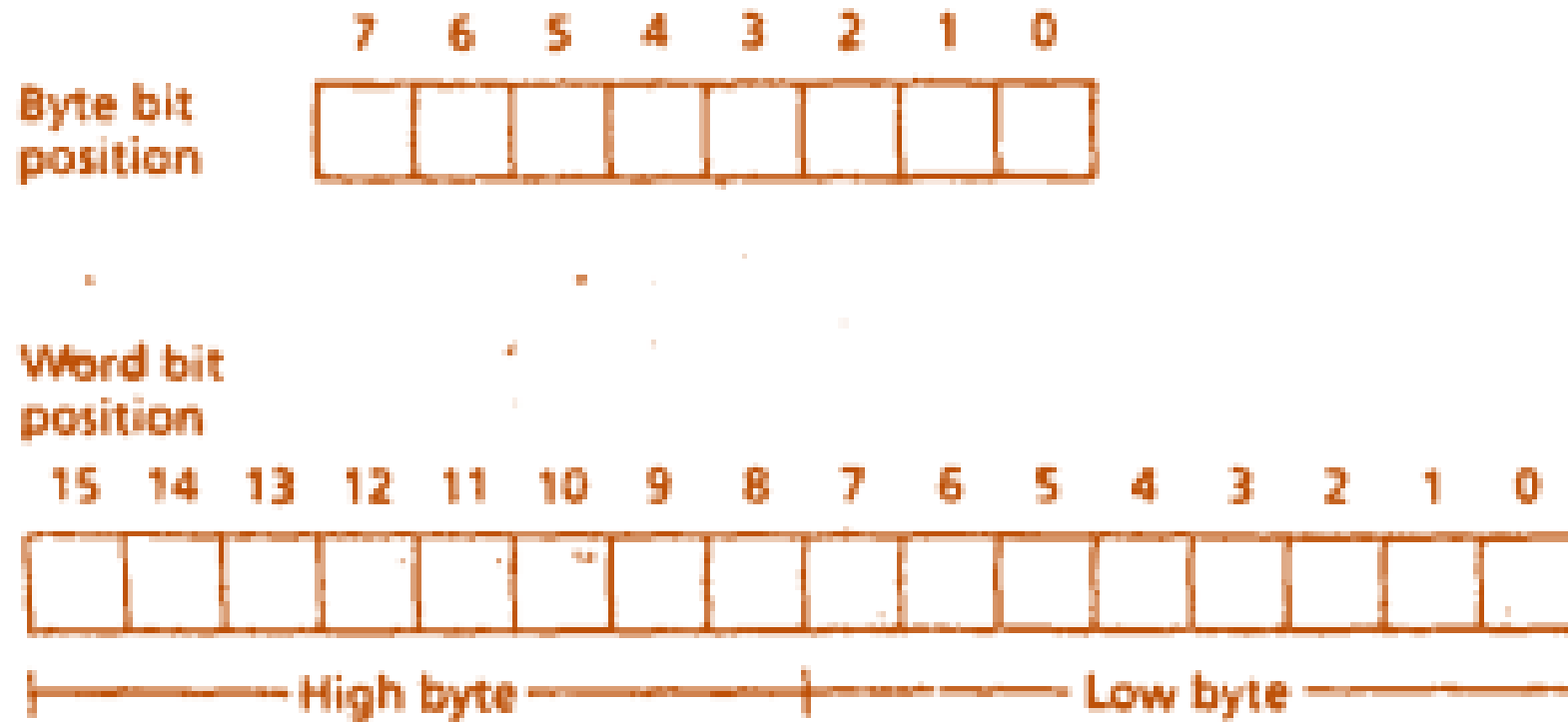
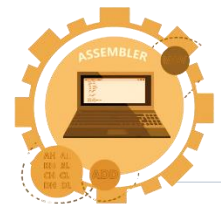


Figure 1.4



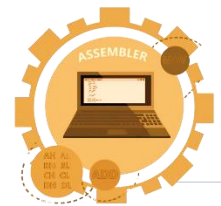
Word Structure:

- In a word, bits 0 to 7 form the low byte, and bits 8 to 15 form the high byte.
- For a word stored in memory, the low byte is from the memory byte with the lower address, and the high byte is from the memory byte with the higher address.



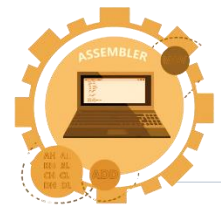
Memory Operations:

- Processor can perform two operations on memory: read (fetch) and write (store).
- In a read operation, the processor gets a copy of the data without altering the original contents.



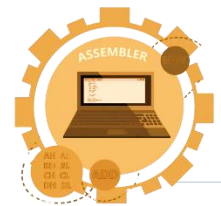
Two Types of Memory Circuits:

- Random Access Memory (RAM): Read and write operations possible.
- Read-Only Memory (ROM): Read-only; contents are initialized and cannot be changed.



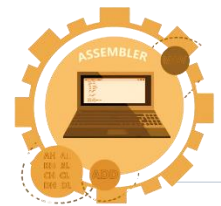
RAM Usage:

- Program instructions and data are loaded into RAM.
- RAM contents are lost when the machine is turned off; need to be saved on disk or printed.



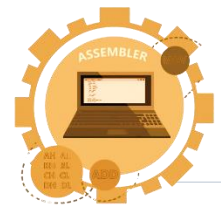
ROM Usage:

- ROM retains values even when power is off.
- Used by computer manufacturers to store system programs (firmware).



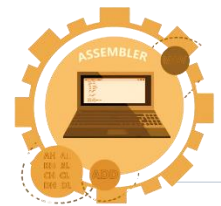
Communication with Memory and I/O Circuits:

- Processor communicates using signals along buses (connections).
- Three kinds of signals: address, data, and control.
- Three buses: address bus, data bus, and control bus.



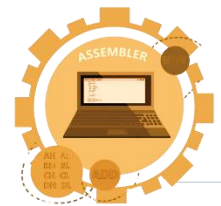
Processor's Role:

- CPU is the brain of the computer.
- Controls the computer by executing programs stored in memory.
- Programs are in machine language, a sequence of 0's and 1's.
- CPU's instruction set is unique for each CPU.



Instruction Execution:

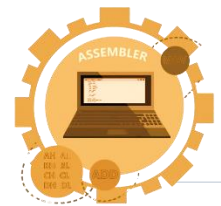
- Instructions performed by the CPU are its instruction set.
- Machine language instructions designed to be simple for cost efficiency.
- Complex tasks performed by computers are sequences of basic operations.





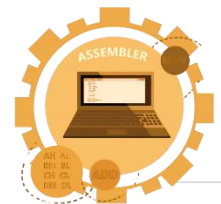
Example: Intel 8086 Microprocessor:

- Organization consists of two main components: execution unit and bus interface unit.



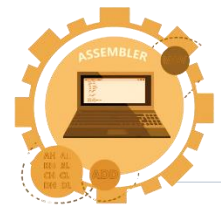
Execution Unit (EU):

- Purpose: Execute instructions.
- Contains Arithmetic and Logic Unit (ALU) for arithmetic and logic operations.
- Operands stored in circuits called registers.
- Eight registers: AX, BX, CX, DX, SI, DI, BP, SP.
- Temporary registers for ALU operands and FLAGS register for computation results.



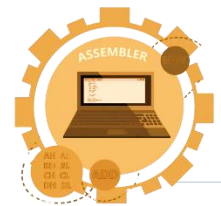
Bus Interface Unit (BIU):

- Facilitates communication between EU and memory/I/O circuits.
- Transmits addresses, data, and control signals on buses.
- Registers: CS, DS, ES, SS, IP (instruction pointer).
- IP holds the address of the next instruction.



EU and BIU Interaction:

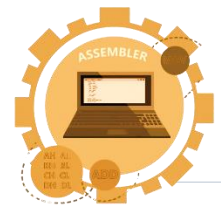
- EU executes instructions, while BIU prefetches up to six bytes of the next instruction.
- Operation called instruction prefetch to speed up the processor.
- BIU can suspend prefetch for communication with memory or peripheral





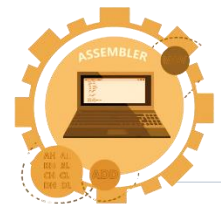
I/O Devices and Circuits:

- I/O devices connected through I/O circuits.
- I/O circuits have registers (I/O ports) for data and control commands.
- I/O ports have addresses and are connected to the bus system.



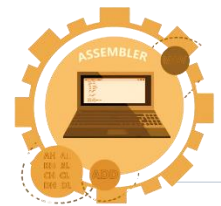
I/O Port Function:

- I/O ports act as transfer points between CPU and I/O devices.
- Input data from an I/O device is sent to a port for CPU to read.
- Output data from CPU is written to an I/O port, which then transmits data to the I/O device.



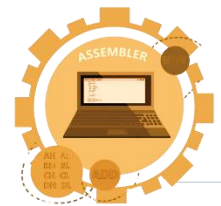
Serial and Parallel Ports:

- Data transfer can be serial (1 bit at a time) or parallel (8 or 16 bits at a time).
- Parallel ports have more wiring connections and can transfer more data at once.
- Serial ports are slower but are suitable for slower devices like keyboards.
- Fast devices like disk drives typically use parallel ports.
- Some devices, like printers, can connect to either serial or parallel ports.



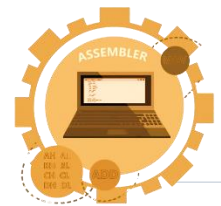
Machine Instruction Components:

- Op code (operation type) and operands.
- Operands often specified as memory addresses.



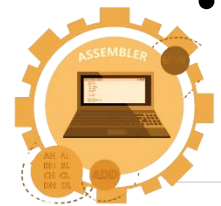
Fetch-Execute Cycle Steps:

- **Fetch:**
 - Fetch an instruction from memory.
 - Decode the instruction to determine the operation.
 - Fetch data from memory if necessary.
- **Execute:**
 - Perform the operation on the data.
 - Store the result in memory if needed.



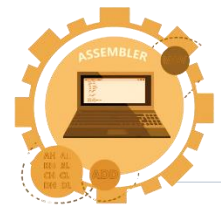
Execution Steps:

- Fetch:
 - Fetch instruction from memory.
 - Add four to the Instruction Pointer (IP) for the next instruction's address.
- Decode:
 - Decoder circuit in the Execution Unit (EU) decodes the instruction.
- Fetch Data:
 - EU informs Bus Interface Unit (BIU) to get data from memory.
 - Memory read request for the specified address.
- Execute Operation:
 - Contents of holding register and AX register sent to Arithmetic and Logic Unit (ALU).
 - ALU performs the required addition and holds the sum.
- Store Result:
 - EU directs BIU to store the sum at a specified memory address.
 - Memory write request for the specified address, with the sum sent over the data bus.



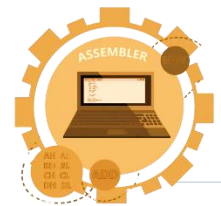
Clock Control:

- Clock circuit generates pulses, controlling processor operations.
- Clock period is the time between two pulses.
- Clock rate or speed is measured in megahertz (MHz).
- One megahertz equals 1 million cycles (pulses) per second.



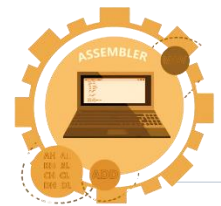
Timing:

- Clock pulses activate computer circuits.
- Each step in the fetch and execute cycle requires one or more clock periods.
- Clock speed influences the processor's operation; higher speeds may lead to faster processing.
- Each processor has a maximum rated clock speed beyond which it may not function properly.



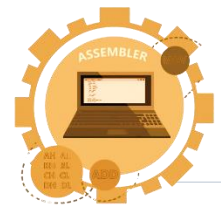
Programming Languages: Machine Language:

- CPU executes machine language instructions (bit strings).
- Example IBM PC machine language program provided.



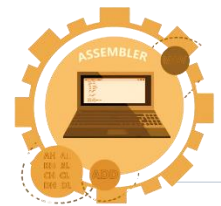
Assembly Language:

- A more convenient language than machine language.
- Uses symbolic names for operations, registers, and memory locations.
- Example program in IBM PC assembly language provided.
- Programs in assembly language need translation to machine language by an assembler.



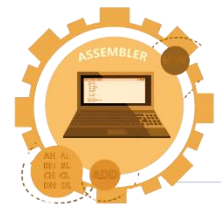
High-Level Languages:

- Developed for ease of programming.
- Closer to natural language, easier to read and understand.
- Examples include FORTRAN and C.
- Compiler needed to translate high-level language programs into machine code.
- Compilation more involved than assembling due to complex expressions and natural language commands.



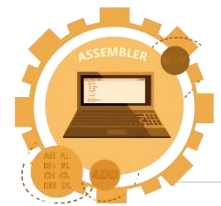
Advantages of High-Level Languages:

- Easier conversion from natural language algorithm to program.
- Easier to read and understand compared to assembly language.
- Fewer statements needed than in assembly language for equivalent programs.
- Portable across different machines with compilers for that language.



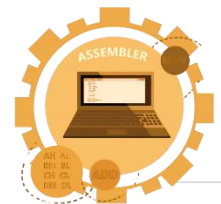
Advantages of Assembly Languages:

- Efficiency: Assembly language programs are faster and shorter.
- Some operations (e.g., reading/writing specific memory locations, I/O ports) are easier in assembly language.
- Allows programmers to mix assembly language with high-level languages.



Importance of Learning Assembly Language:

- Provides insight into how the computer "thinks" and operates.
- Understanding details of compiled machine language programs.
- Helps comprehend runtime behavior and potential issues.
- Studying assembly language specific to one system (e.g., IBM PC) applies to other assembly languages.





THANK YOU!

Do you have any questions?

ENGR. FIAZ KHAN



LECTURER COMPUTER SCIENCE

TELF/EF SET / ACEPT CERTIFIED

MICROSOFT ACADEMY TRAINER

ORACLE CERTIFIED PROFESSIONAL

GOOGLE CERTIFIED PROFESSIONAL

Lahore, Pakistan



+92-320-7617-093



muhammad.fiaz@cs.uol.edu.pk



<https://www.linkedin/in/fiazofficials>

